Daffodil International University

Department of Computer Science and Engineering

Proposal Approval Form

(To be attached to the FYDP Title Phase Evaluation Report)

I. Project Information			
Course Code:	CSE499	**Course Title:**	Final Year Design Project

II. Student Information			
Student Name	**Student ID**	**Batch**	**Program & Email**

III. Proposed Title:	**Nirapod Parking: A Community-Driven Residential Parking Marketplace Using Java Spring Cloud Microservices, Angular, Keycloak OAuth2, and Geospatial Proximity Indexing**
IV. Updated Title:	

This is the approval for the Final Year Design Project (FYDP) proposal and title submitted by the above-mentioned student who is from the Department of CSE under my supervision. Upon thorough review and evaluation, it is my professional opinion that the proposed project title [mentioned in Sec III and IV] is both relevant and aligned with the academic goals and standards of our department. I (Supervisor) have endorsed the proposed proposal and title. Under our department's guidelines, the student has fulfilled all the necessary prerequisites and will attach this Proposal Approval Form to their FYDP Title Phase Evaluation Report.

Supervisor Name:	
Designation:	

Approved by: _____ **Supervisor Signature with Date**

V. Detailed Proposal

1. Objective

The main objective of this project is to develop Nirapod Parking, an easy-to-use web platform designed for crowded urban centers like Dhaka and Chattogram. The platform connects Vehicle Owners (commuters driving cars or motorcycles) who need safe, temporary parking with Parking Spot Owners (residential flat owners or authorized residents) who have empty, unused garage spaces. Through our system, drivers can easily find and reserve parking for a few hours, for a full workday, overnight, or on a monthly subscription basis.

In busy commercial and residential areas such as Dhanmondi, Banani, Motijheel, and Agrabad, finding parking is an exhausting everyday struggle. Drivers frequently spend 20 to 45 minutes circling congested streets, wasting expensive fuel and worsening citywide traffic jams. Because our cities have almost no dedicated public parking facilities, vehicle owners are forced to park on roadside streets where their cars and bikes regularly suffer body scratches, stolen parts (such as side mirrors, helmets, or batteries), and police towing fines. At the same time, thousands of private residential parking bays sit completely empty during the day while flat owners are away at work. Many other spots stay vacant permanently because a family does not own a vehicle, an expatriate flat owner lives abroad, or a tenant simply has no personal car or bike. Currently, there is no organized or trustworthy way for residential flat owners to share these idle spaces with daily commuters.

Nirapod Parking solves this problem by providing a completely software-based sharing platform that requires zero expensive hardware or gate sensors. Vehicle Owners can search for available nearby spots on an interactive clustered map, book parking according to their preferred schedule, and receive a digital booking pass with their vehicle license plate number. To ensure building security, an Admin verifies the Parking Spot Owner's National ID, property allotment papers, and garage photos before any spot goes live, while the building's Security Guard can quickly verify incoming vehicles at the gate simply by matching the license plate number on the digital pass.

2. Motivation

Over the last few years, the sharing economy has transformed urban mobility across Bangladesh. Platforms like Pathao and Uber proved that solving city transportation does not require adding thousands of new vehicles, but rather making smart use of empty seats in cars and motorbikes already on the road. Similarly, platforms like Airbnb proved that vacant residential rooms could solve accommodation shortages without building new hotels. Nirapod Parking is inspired by this exact concept: bringing the sharing economy to Bangladesh's untouched parking sector. Every day in Dhaka, two-lane roads are blocked by parked vehicles, while right behind the boundary gates of adjacent residential buildings, dozens of covered parking spots sit completely vacant. Unlocking this hidden, private capacity is the driving inspiration behind our project.

Anyone who travels through Dhaka or Chattogram experiences the intense daily anxiety of finding parking. Vehicle drivers heading to universities, offices, or medical appointments face constant mental stress, frequently arriving late because they must circle blocks in gridlocked traffic. Once parked on the street, commuters spend their entire day worrying about vehicle damage, theft, or police towing fines. At the same time, municipal authorities cannot build new multi-story public parking lots in crowded commercial hubs because there is simply no open land available, and land acquisition costs are astronomical. Nirapod Parking provides an immediate, zero-cost alternative by taking parked vehicles off narrow streets and moving them into existing residential garages, easing road congestion without requiring any government spending or public land acquisition.

When reviewing smart parking research, almost all published academic papers focus heavily on complex IoT hardware-such as ultrasonic ground sensors, automated boom barriers, and dedicated surveillance cameras. While these systems work in commercial shopping malls, they are completely impractical and cost-prohibitive for ordinary apartment buildings in South Asia. Furthermore, existing research largely ignores community-based parking sharing models that accommodate local apartment realities, such as locked gates, on-duty Security Guards, and multiple flat owners sharing a single compound. Nirapod Parking addresses this research gap by demonstrating a purely software-based, zero-hardware marketplace tailored specifically for residential communities, while providing middle-class flat owners with a steady source of passive monthly income (estimated at -3,000 to -8,000) from an otherwise wasted space.

3. Background Study

To prepare for this project, we reviewed existing academic literature and real-world parking solutions both globally and locally in Bangladesh. Previous work in this domain can be divided into three main areas:

First, a large body of university research papers focuses on automated parking systems built with physical hardware. Researchers commonly install ultrasonic sensors, magnetic detectors, or camera-based image processing systems at each parking slot to track occupancy, connecting them to cloud servers using microcontrollers like Arduino or Raspberry Pi. These studies have established an important finding: giving drivers real-time slot availability and the ability to reserve spots in advance can reduce cruising and searching time by up to 30% to 50%. However, these systems are almost always evaluated in controlled lab settings or commercial shopping malls; in real life, buying, wiring, and maintaining expensive hardware in ordinary residential apartment buildings across Dhaka is financially and practically impossible.

Second, in Western countries, commercial platforms such as JustPark in the UK and SpotHero in the USA popularized peer-to-peer parking sharing by letting homeowners rent out their private driveways through mobile apps. These platforms proved that sharing private parking spaces works effectively, generates passive income for property owners, and takes cars off busy roads. However, these apps were designed for Western single-family suburban houses with open, unmonitored driveways and direct credit card billing. In Dhaka and Chattogram, residential buildings are multi-story apartment complexes with locked gates, on-duty Security Guards, and multiple residential flat owners sharing the same ground-floor garage. Foreign apps offer no mechanism to verify flat ownership, handle building security, or support gate security staff.

Third, in Bangladesh, platforms like Pathao, Uber, and Jatri successfully modernized ride-sharing and bus ticketing, but vehicle parking was completely left out of this digital transition. Currently, parking in cities like Dhaka relies either on commercial shopping mall basements, informal roadside parking with cash slips, or handwritten paper 'Garage To-Let' notices pasted on building boundary walls. These manual paper notices are slow, unorganized, completely unverified, and only work for long-term monthly rentals. They cannot help a commuter who needs a safe spot for just two hours during a meeting or overnight for an event. Nirapod Parking steps in to bridge these gaps by providing a modern web platform built specifically around local residential customs.

4. Gap Analysis

Although previous academic papers and commercial apps have addressed parking, they break down when applied to the realities of crowded residential neighborhoods in Bangladesh. Our analysis identifies four critical practical and technical barriers:

1. The Hardware and Cost Barrier: Academic smart parking models require dedicated physical sensors, electrical wiring, and automated boom barriers at every parking bay. Installing and maintaining costly hardware across private residential apartment buildings in Dhaka is financially unrealistic for ordinary flat owners.

2. The Building Structure and Trust Barrier: International apps like JustPark assume open, single-family driveways. In Dhaka, residential buildings have locked gates, and multiple flat owners share the same garage. Letting unverified strangers into the building creates serious security concerns for residents. Existing foreign apps and local paper signs provide zero background verification.

3. The Gate Usability Barrier: Smart city research papers assume automated electronic barrier gates. In Bangladesh, building entry is managed by human Security Guards (Darwans). Systems that require guards to use complex smartphone apps or scan digital codes fail in practice due to usability and technical friction.

4. The UI/UX Map Collision Barrier: In multi-story Dhaka buildings where three or four different flat owners list their parking spots, standard map applications drop multiple marker pins at the exact same GPS coordinate. This causes pins to overlap, creating visual clutter that makes individual spots impossible for Vehicle Owners to click or select.

Nirapod Parking overcomes these barriers through a software-first approach tailored to local needs. The platform operates entirely through standard web browsers using device GPS and SQL distance formulas, completely eliminating the need for expensive physical sensors or barrier gates. To establish trust and maintain building security, an Admin reviews the Parking Spot Owner's National ID, flat allotment or tax slip, and a clear photo of the parking slot before any listing is published. Furthermore, when a booking is confirmed, the system generates a digital pass showing the vehicle's license plate number so that the building's Security Guard can verify incoming cars and motorbikes with a quick visual check at the gate, without needing a smartphone or app. On the user interface, Leaflet marker clustering groups multiple spots in the same building into a single clean pin that smoothly expands when clicked, while our flexible booking engine supports hourly, daily, overnight, and monthly reservations for both cars and motorcycles.

Through this combination of features, the project directly addresses the disconnection between chaotic street parking and thousands of idle residential garage spaces across Dhaka and Chattogram. By providing a verified, low-cost digital bridge between Vehicle Owners and residential flat owners, it eliminates street parking dangers (vehicle damage, parts theft, and police towing fines) and road congestion, while allowing flat owners to earn steady supplemental income from empty garage spaces.

5. Methodology

We will execute this project using an Agile incremental development methodology divided into three clear development stages:

* **Stage 1 (Core MVP):** Setting up the distributed microservices backend infrastructure, centralized user authentication, and basic spot listing and reservation workflows.
* **Stage 2 (Map Search & Security):** Integrating an interactive Leaflet map with marker clustering, GPS proximity search, and the Admin KYC verification portal.
* **Stage 3 (Testing & Evaluation):** Conducting performance benchmarking, concurrency load tests to prevent double-booking, and refining responsive UI styling.

By structuring the backend into independent microservices, each service can be developed, tested, and scaled on its own without affecting the rest of the application.

Techniques & Tools (Technical Stack)

* **Backend Framework:** Java 25 & Spring Boot 4.x for developing lightweight, high-performance REST microservices.
* **Microservices Architecture:** API Gateway for centralized routing, rate limiting, and JWT validation; Discovery Server for dynamic registration; and Declarative REST Clients for inter-service communication.
* **Database-per-Service:** MySQL 8.x with dedicated databases (user_db, parking_db, booking_db, and keycloak_db) managed via Spring Data JPA and Hibernate.
* **Identity Management:** Keycloak (OAuth2 / OpenID Connect) supporting a unified dual-role model (Vehicle Owner and Parking Spot Owner).
* **Frontend & Styling:** Angular 19 with Signals state management and SCSS for clean, responsive styling across mobile and desktop.
* **Mapping & Algorithms:** Leaflet.js with OpenStreetMap, Haversine spherical distance formulas in SQL, and Leaflet marker clustering for multi-flat buildings.

Data Collection & Processing

Because this is a software development project, our data collection consists of three practical datasets:

* **Real Geographic Coordinates:** GPS coordinates for popular commercial and residential hubs in Dhaka (Dhanmondi, Banani, Mirpur, Uttara) collected via OpenStreetMap and Google Maps.
* **Synthetic Booking Schedules:** Simulated datasets representing concurrent booking requests, peak morning/evening schedules, and vehicle types (cars and motorbikes).
* **Sample KYC Documents:** Anonymized test NID cards, dummy property allotment slips, and garage photos for testing the Admin verification workflow.

All incoming data will be validated on both the Angular frontend and backend microservices before saving into respective MySQL databases.

Data Analysis & Evaluation Metrics

* **Concurrency & Zero Double-Booking:** Simulating simultaneous booking attempts with Apache JMeter to verify a 0% double-booking collision rate through database transactions.
* **Geospatial Search Latency:** Benchmarking SQL Haversine query response times across 1 km, 2 km, and 5 km radii to ensure sub-second results.
* **API Gateway Throughput:** Measuring round-trip latency across the API Gateway under concurrent loads to ensure smooth frontend responsiveness.

FIGURE 1: NIRAPOD PARKING - DISTRIBUTED MICROSERVICES ARCHITECTURE (A4 FULL PAGE)

Java 25 | Spring Boot 4.x | Angular 19 (Signals & SCSS) | Keycloak OAuth2 | MySQL Database-per-Service

1. PRESENTATION TIER: ANGULAR 19 (STANDALONE COMPONENTS & SIGNALS)

Vehicle Owner View

Split-view card feed + Leaflet map, GPS proximity autodetect, digital pass generation

Spot Owner (Host) View

Listing wizard, 3-point KYC upload (NID, Tax, Photo), -3k--8k/mo revenue analytics

Admin & Guard Portal

KYC verification workbench, vehicle license plate check at gate (zero guard barrier)

2. EDGE GATEWAY & IDENTITY TIER (PORTS 8080 & 8180)

Spring Cloud Gateway (:8080)

Central single entry point, Netty non-blocking, RS256 token filter, URI route predicates

Keycloak IAM Server (:8180)

OAuth2 / OIDC server, RS256 JWT cryptographic signing, dual-role RBAC, keycloak_db

Rate Limiter & Security

Token bucket DDoS protection, CORS security policy, coordinate scraping defenses

3. SERVICE DISCOVERY & DECLARATIVE RPC (NETFLIX EUREKA & FEIGN)

Netflix Eureka Server (:8761)

30s dynamic heartbeat registry, zero hardcoded IPs, auto-eviction of dead instances

Spring Cloud OpenFeign RPC

Synchronous declarative REST inter-service clients, Resilience4j circuit breakers

4. APPLICATION CORE MICROSERVICES TIER (SPRING BOOT 4.X - JAVA 25)

User & KYC Service (:8081)

User profile, vehicle registry (BRTA plate), 3-point KYC state machine, audit log

Parking Spot Service (:8082)

Garage metadata, SQL Haversine spherical distance query, tiered hourly/monthly rates

Booking Service (:8083)

ACID row locks (0% collision), digital pass engine, @Scheduled 5-min overstay poller

5. PERSISTENCE TIER: STRICT DATABASE-PER-SERVICE ISOLATION (MYSQL 8.X)

user_db (:3306)

Exclusive schema: users, vehicles, kyc_docs, audit_log

parking_db (:3306)

Exclusive schema: parking_spots, pricing_plans, spatial_idx

booking_db (:3306)

Exclusive schema: bookings, digital_passes, revenue_ledger

keycloak_db (:3306)

Exclusive schema: realm, user_entity, credentials, roles

6. EXTERNAL INTEGRATIONS & SYSTEM SPECIFICATIONS

External Services

OpenStreetMap Tile CDN, HTML5 Browser GPS (navigator.geolocation), Google Maps intent

System Quality Metrics

0% double-booking collision, sub-second search latency (<250ms), pure software zero-IoT

Figure 1: Nirapod Parking - Distributed Microservices Architecture & Database-per-Service Model (A4 Full-Page Edition)

FIGURE 2: NIRAPOD PARKING - END-TO-END OPERATIONAL WORKFLOW SEQUENCE (A4 FULL PAGE)

Flows 1-5: OIDC Auth, 3-Point Host KYC, Leaflet Proximity Search, Gate Arrival Check & Overstay Poller

FLOW 1: CENTRALIZED AUTHENTICATION & UNIFIED DUAL-ROLE REGISTRATION

Step 1.1: Client Input

User submits credentials on Angular 19; unified account serves driver and host.

Step 1.2: Gateway Proxy

Spring Cloud Gateway (:8080) validates rate limit, proxies request to Keycloak (:8180).

Step 1.3: RS256 JWT Token

Keycloak validates credentials against keycloak_db, signs RS256 JWT with dual roles.

Step 1.4: Profile Init

Angular Signals store JWT token; async Feign initializes profile in user_db (:8081).

FLOW 2: GARAGE SPOT LISTING, 3-POINT KYC VETTING & ADMIN APPROVAL

Step 2.1: Host Listing Wizard

Host drops Leaflet GPS pin, enters dimensions, and sets hourly/daily/monthly rates.

Step 2.2: 3-Point KYC Upload

Host uploads NID front/back, flat allotment tax slip, and 3 garage space photos.

Step 2.3: Admin Vetting Portal

Admin reviews documents in review portal; validates NID against apartment ownership.

Step 2.4: Activation & Publish

On approval, status = VERIFIED; Parking Service activates spot in parking_db.

FLOW 3: PROXIMITY SEARCH, LEAFLET CLUSTERING & TRANSACTIONAL RESERVATION

Step 3.1: Browser GPS Proximity

Browser GPS captures coordinates via HTML5 navigator.geolocation without app install.

Step 3.2: SQL Haversine Query

Parking Service executes SQL Haversine query across 1km, 2km, 5km radius in <250ms.

Step 3.3: Clustered Map & Feed

Leaflet.markercluster groups multi-spot buildings; split card feed shows prices.

Step 3.4: ACID Pessimistic Lock

Booking Service acquires row-level lock; 0% collision rate; generates digital pass.

FLOW 4: GATE ARRIVAL, SECURITY GUARD VERIFICATION & ENTRY CHECK-IN

Step 4.1: Google Maps Handoff

Driver taps Navigate; one-tap deep link launches Google Maps turn-by-turn guidance.

Step 4.2: Guard Plate Match

Vehicle reaches gate; driver displays pass showing plate; guard visually matches plate.

Step 4.3: Optional Web Lookup

Security guard can enter 6-digit pass code in lightweight web portal to verify booking.

Step 4.4: Gate Open & Check-in

Gate barrier opens; vehicle parks; driver confirms check-in; duration timer begins.

FLOW 5: AUTOMATED OVERSTAY MONITORING, CHECK-OUT & SETTLEMENT

Step 5.1: 5-Min Cron Poller

Spring @Scheduled poller checks active reservations; flags overstay if time exceeded.

Step 5.2: Overstay Surcharge

System calculates base reservation fee plus overstay penalty; dispatches SMS alert.

Step 5.3: Exit Clearance

Guard checks exit pass; barrier opens; vehicle departs residential compound safely.

Step 5.4: Revenue Settlement

Booking completed; earnings credited to host ledger (-3k--8k/mo); mutual ratings.

Figure 2: Nirapod Parking - End-to-End Operational Workflow Sequence (A4 Full-Page Edition)

6. Expected Outcomes

The successful completion of this project will deliver a fully functional, cloud-ready web platform. The primary technical deliverable is an independent microservices backend built with Java 25, Spring Boot 4.x, an API Gateway, a Discovery Server, and strict Database-per-Service isolation on MySQL (user_db, parking_db, booking_db). On the frontend, users will interact with a responsive Angular 19 single-page application styled with SCSS, featuring an interactive clustered map, an area-filtered card feed, and dedicated dashboards for both booking and listing management. The platform also integrates centralized Keycloak OAuth2 authentication with a unified dual-role model, an Admin review portal for 3-point KYC document vetting, and digital booking passes that display vehicle license plate numbers for quick verification by building Security Guards.

Beyond the working software, this project makes a tangible academic and technical contribution to urban software engineering. It proves that community parking in high-density developing cities can be solved effectively through software alone-using standard browser GPS, SQL distance calculations, and map marker clustering-without requiring costly IoT sensor hardware or automated boom barriers. Furthermore, it serves as a practical, production-grade reference architecture for undergraduate engineering students, demonstrating best practices in distributed database isolation, declarative inter-service REST communication, and transactional integrity under load.

In real-world terms, the project achieves practical results for all three major urban stakeholders. For Vehicle Owners, it eliminates 20 to 45 minutes of wasted driving time per trip, ensures vehicle safety from roadside damage or theft, and provides flexible booking options (hourly, daily, overnight, and monthly subscriptions) for both cars and motorcycles. For residential flat owners, it turns an otherwise idle private garage bay into a steady source of passive monthly income (estimated at -3,000 to -8,000). For the city, it clears road lanes in congested hubs like Dhanmondi and Banani without requiring any public land or municipal budget. Finally, rigorous concurrency stress testing with Apache JMeter will confirm a 0% double-booking collision rate and sub-second search latency, verifying that the system remains reliable during peak demand.

7. Conclusion

Nirapod Parking presents a realistic, community-driven software solution to the severe urban parking crisis in crowded cities like Dhaka and Chattogram. Rather than waiting for expensive municipal parking structures or deploying unaffordable IoT sensor hardware, the platform unlocks the hidden capacity inside private residential buildings. It connects Vehicle Owners who desperately need safe, temporary parking with Parking Spot Owners (residential flat owners) whose garage spaces sit empty during the day, overnight, or permanently.

In land-scarce cities where open land is virtually non-existent and land acquisition costs billions, building new multi-story parking plazas is practically impossible. Nirapod Parking provides an immediate, zero-cost alternative by optimizing infrastructure that is already built. It removes parked vehicles from narrow streets to ease traffic flow, protects commuters from roadside theft and police towing fines, and provides middle-class apartment owners with effortless passive income.

What makes Nirapod Parking uniquely relevant is that it bridges modern distributed software engineering with local social realities. By combining a reliable microservices backend (Java 25, Spring Boot 4.x, MySQL, Keycloak) and an Angular 19 frontend with practical local features-such as 3-point host KYC, Leaflet map clustering for multi-flat buildings, and simple license plate verification for building Security Guards-the project delivers a feasible, defensible, and high-impact capstone solution for Bangladesh.